

DEEP LEARNING FROM SCRATCH

A Guide for Self-Learners

Episode V

The Rise of Intelligence

*You've always wanted to understand how neural networks really work.
But every resource is either too shallow or a 200-page textbook.*

This is neither.

The math is tough, but you only need to do it once.

Why this episode?

We go deep into the real equations.

Forward propagation, backpropagation, gradients.

No shortcuts. No hand-waving. Just honest computation.

“What I cannot create, I do not understand.”

– Richard Feynman

Pierre Chambet

March 2026

Contents

1	Theory of Neural Networks	3
2	Machine Learning Approach	3
3	Deep Learning Approach	4
4	Let's Go	4
4.1	What Is a Layer?	5
4.2	Neuron by Neuron	5
4.3	The Output Equation — Matrix Form	5
5	Activation Function	6
5.1	Visual Summary	6
5.2	Very Important	7
6	Adding Neurons in the Layer	7
7	A Layer is a Collection of Neurons	8
8	Adding a Second Layer	9
9	Second Layer Computation	10
10	Adding More Neurons to the Second Layer	10
10.1	Visual Recap	11
11	Adding a Layer	12
12	Output Layer and Final Output	12
13	Summary	13
14	Forward Propagation	13
15	Training via Backpropagation	14
15.1	Goal: Compute the Gradients	14
15.2	Summary of the Network Functions	15
16	Important	15
17	Gradients for the Layer Just Before the Output	16
18	For the First Layer	16
19	Full Gradient Expression for the First Layer	17
20	Summary of the Gradient Equations	18

21 Let's Get to the Point	19
21.1 Computing dZ_2	19
21.1.1 Final Expression for dZ_2	21
21.2 Computing $\frac{\partial \mathcal{L}}{\partial W_2}$ and $\frac{\partial \mathcal{L}}{\partial b_2}$	21
21.3 Computing dZ_1	22
21.4 Computing $\frac{\partial \mathcal{L}}{\partial W_1}$ and $\frac{\partial \mathcal{L}}{\partial b_1}$	23
22 Summary of the Gradients	24

What You'll Learn in This Episode

After these 25 pages, you won't just know what a neural network is — you'll know *how it thinks, how it learns, and how to build one yourself*.

1. Why a single neuron isn't enough — and what deep learning does about it
2. How to build a **layer** of neurons using matrix operations: $Z = Wx + b$
3. What an **activation function** does and why it matters
4. How to stack layers into a **deep neural network**
5. The complete **forward propagation** pipeline
6. The full derivation of **backpropagation** — every gradient, step by step
7. The final gradient equations you'll implement in code

1 Theory of Neural Networks

In the previous episode, we trained a single neuron. It worked — on simple data. But it hit a wall: one neuron can only draw one straight line. That’s not enough for real problems.

Today, we break through that wall.

What’s Ahead

This episode is about the **real math** behind neural networks. It’s dense. It’s honest. And you only need to do it **once**.

After this, you’ll understand what every framework hides from you — and that understanding is what separates someone who uses tools from someone who can *build* them.

Remember — a data scientist is not a mathematician. They rarely compute things by hand. Instead, they rely on libraries to do the heavy lifting. So, your job is simple: understand it deeply, once and for all. After that, you’re good.

And that’s the beauty of it: very few people truly understand what’s going on behind the scenes. But those who do? They’re the ones capable of designing their own models and architectures.

Let’s Begin

No more talking — let’s dive into the theory of neural networks.

We’ve already encountered the concept of a neuron through the perceptron model.

- It takes multiple inputs,
- Applies weights,
- Produces an output using a threshold activation function.

This is the foundation. And yes, the perceptron can classify data into two categories — but it quickly reaches its limits. **It’s not powerful enough** for real-world complexity.

2 Machine Learning Approach

To overcome this, traditional **machine learning** applies a clever workaround. We enrich the input space with manually engineered features: things like x_1^2 , x_2^2 , x_1x_2 , and so on.

This transforms a linear model into a more flexible one — usually polynomial. The process is called *feature engineering*.

Feature Engineering

The art of creating new features from existing ones — based on domain knowledge, intuition, and a bit of trial-and-error.

It works — but it’s manual, tedious, and not scalable to complex data.

3 Deep Learning Approach

Deep learning asks a new question:

What if... we connected neurons together?

Instead of manually building features, we let the network *learn them itself*.

How?

We stack layers of neurons — one after the other — forming what we call a **deep neural network**. Each layer learns to extract more abstract, more meaningful representations from the previous one.

This is the essence of deep learning:

Key Idea

Let the model automatically learn the best features by stacking multiple layers of neurons.

Each transformation brings us closer to understanding the data. Each layer does a little more, building up to a powerful representation.

In the end, the model becomes capable of learning incredibly complex functions — **and making remarkably accurate predictions**.

Sounds intense?

Good. That's how you know it's worth it.

We're going to build a neural network from scratch — using math and matrix operations.

4 Let's Go

We're not here to waste time. Let's take the neuron we previously built — and simply **duplicate** it.

We now have two neurons: N_1 and N_2 , placed side by side. And just like that, we've built our first:

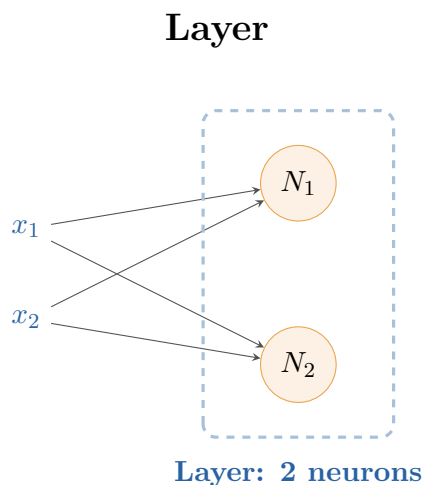


Figure 1: A layer groups multiple neurons — each with its own weights and bias — but sharing the same input.

4.1 What Is a Layer?

A **layer** is simply a collection of neurons that operate *in parallel*. They all receive the same input vector, but each computes something slightly different.

- Each neuron has its own weight vector and bias.
- Each neuron produces one output.

So here, with N_1 and N_2 , we'll get two outputs: z_1 and z_2 .

4.2 Neuron by Neuron

Neuron N_1 takes the input (x_1, x_2) and computes:

$$z_1 = W_{11} \cdot x_1 + W_{12} \cdot x_2 + b_1$$

Where:

- W_{11} and W_{12} are the weights of N_1 ,
- b_1 is its bias.

Neuron N_2 does the exact same thing — but with different weights and bias:

$$z_2 = W_{21} \cdot x_1 + W_{22} \cdot x_2 + b_2$$

So even though they process the same input, the result is different — because their parameters are different.

This is the power of layers: many neurons, one input, multiple outputs.

4.3 The Output Equation — Matrix Form

Now let's write all of this in a single, elegant expression:

$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Or, more compactly:

$$Z = Wx + b$$

Notation Guide

- x is the input vector (shared by all neurons),
- W is the weight matrix — each row is a neuron,
- b is the bias vector,
- Z is the output vector of the layer.

Key Insight

The matrix equation $Z = Wx + b$ allows us to compute the output of the whole layer in one step — it's the vectorized form of everything we just did manually.

5 Activation Function

We're almost there. Now that our neurons have computed their raw outputs z_1 and z_2 , we apply a final transformation:

The activation function.

Why? Because we want to introduce **non-linearity** into our model — so it can learn complex patterns, not just straight lines.

Sigmoid Activation Function

The sigmoid is a classic choice:

$$a(z) = \frac{1}{1 + e^{-z}}$$

It maps any real number to a value between 0 and 1. Perfect when we want to interpret outputs as probabilities.

We apply the sigmoid to each neuron's output individually:

$$A = a(Z) \Rightarrow \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} a(z_1) \\ a(z_2) \end{bmatrix} = \begin{bmatrix} \frac{1}{1+e^{-z_1}} \\ \frac{1}{1+e^{-z_2}} \end{bmatrix}$$

Here, a_1 and a_2 are the **activated outputs** of neurons N_1 and N_2 .

Key Insight

Each neuron computes its own linear combination: $z = Wx + b$ Then applies an activation: $a = \sigma(z)$ The result: nonlinear, learnable, expressive outputs.

5.1 Visual Summary

Here is what we've just built — a layer of two neurons with activation functions:

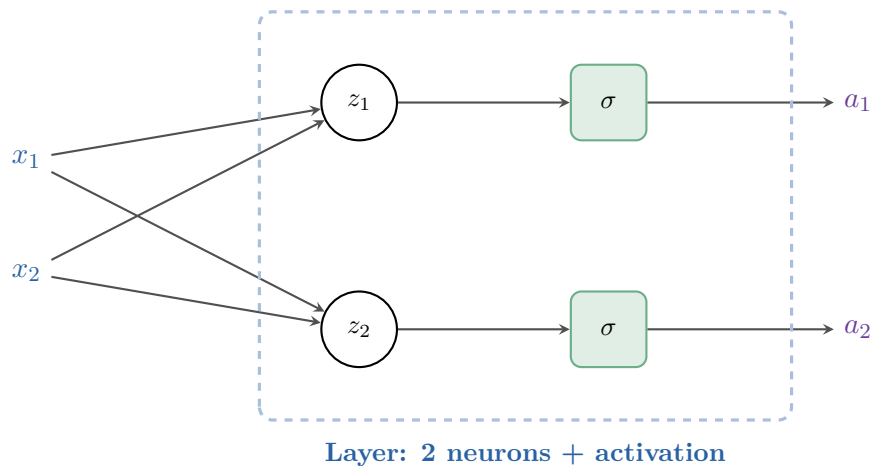


Figure 2: Each neuron computes $z = Wx + b$, then applies $\sigma(z)$ to produce its final output a .

5.2 Very Important

The outputs of the activation function are the true outputs of the layer.

In our case, we now have two activated outputs:

$$\boxed{a_1 \quad \text{and} \quad a_2}$$

These form the output vector of the first layer.

6 Adding Neurons in the Layer

Let's say we want more power. What's stopping us from adding more neurons to the layer?

Answer

Nothing. We just add more rows to the weight matrix, and more entries in the bias vector.

Here's what that looks like:

$$W = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Same input x , same formula:

$$Z = Wx + b$$

Now, the output becomes:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Each new row corresponds to a new neuron, with its own weights and bias. The logic remains exactly the same.

We can add 3, 10, 100 neurons — as many as we want — and the equation stays the same.

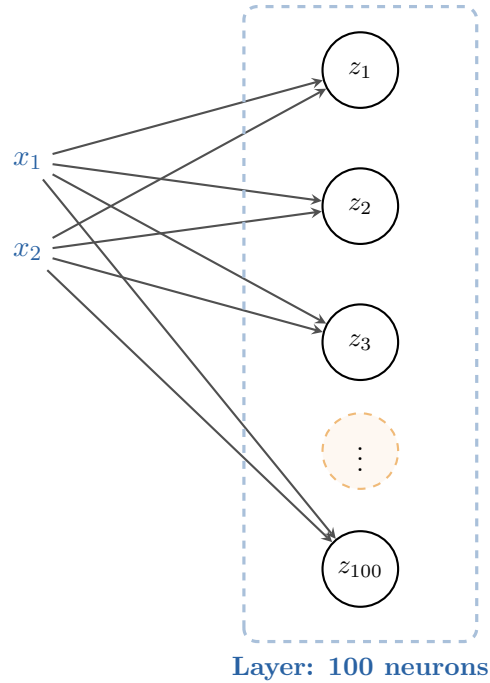


Figure 3: A layer with 100 neurons — each receiving the same input, but with its own weights and bias.

7 A Layer is a Collection of Neurons

Let's take a step back and reframe what a layer really is.

A **layer** is just a collection of neurons working in parallel. It takes an input, and produces multiple outputs — one from each neuron.

Analogy

You can think of a layer as one giant neuron — but made of many smaller neurons, each with its own weights and bias.

For now, we're working with two neurons in our first layer.

We now introduce the notation for layers explicitly. The output of the first layer — after applying the activation — is written as:

$$A = A_1 = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = a(Z_1) = \begin{bmatrix} a(z_1) \\ a(z_2) \end{bmatrix}$$

Where:

- Z_1 is the pre-activation output of the first layer (before sigmoid),
- A_1 is the activated output of the first layer,
- a_1 and a_2 are the outputs of neurons N_1 and N_2 after activation.

8 Adding a Second Layer

Now, something exciting happens.

We're no longer going to feed the raw input x to the layer — we're going to feed it the output of the previous layer: A_1 .

Deep Architecture

- The first layer takes the original input x and produces A_1 .
- The second layer takes A_1 as input and produces A_2 .

Let's keep it simple for now. **We'll add just one neuron in the second layer.** This neuron will:

- Take a_1 and a_2 as inputs,
- Apply its own weights and bias,
- Produce a pre-activation z_3 ,
- Then apply the activation to get a_3 .

$$z_3 = W_{31} \cdot a_1 + W_{32} \cdot a_2 + b_3 \quad \Rightarrow \quad a_3 = \sigma(z_3)$$

So now we have:

$$A_2 = [a_3]$$

Important Insight

Each new layer takes the output of the previous layer as input. This stacking is what gives neural networks their depth — and power.

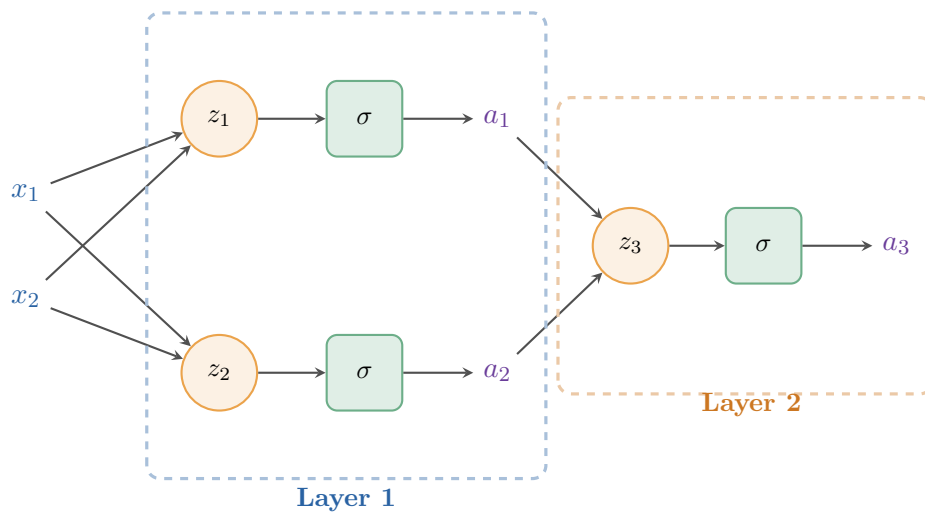


Figure 4: A two-layer neural network: the output of Layer 1 becomes the input to Layer 2.

9 Second Layer Computation

We now know that the output of the first layer is:

$$A_1 = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

Let's feed this into the second layer. This time, we'll work with a single neuron in the second layer.

Layer 2 Computation

The second layer computes:

$$Z_2 = W_2 A_1 + b_2$$

Where:

- W_2 is the weight matrix of the second layer,
- A_1 is the input (from the first layer),
- b_2 is the bias of the second layer.

Then we apply the activation function:

$$A_2 = a(Z_2)$$

Let's look at a concrete example. Suppose we have:

$$W_2 = \begin{bmatrix} w_{21} & w_{22} \end{bmatrix}, \quad b_2 = \begin{bmatrix} b_3 \end{bmatrix}$$

Then:

$$Z_2 = \begin{bmatrix} z_3 \end{bmatrix} = W_2 A_1 + b_2 = \begin{bmatrix} w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} b_3 \end{bmatrix}$$

And the final activated output becomes:

$$A_2 = \begin{bmatrix} a_3 \end{bmatrix} = a(Z_2) = \begin{bmatrix} a(z_3) \end{bmatrix} = \begin{bmatrix} \frac{1}{1+e^{-z_3}} \end{bmatrix}$$

This shows a full forward pass through a 2-layer neural network. From $x \rightarrow A_1 \rightarrow A_2$, the data flows forward — layer by layer — with weights, biases, and activations applied at each stage.

Key Insight

Each layer is a clean block: linear operation ($Wx + b$) followed by a non-linearity ($a(z)$). Stacking these blocks is what builds a deep neural network.

10 Adding More Neurons to the Second Layer

Why stop at one neuron?

We can easily add more neurons to the second layer by:

- adding more **rows** to the weight matrix W_2 ,

- and more **entries** to the bias vector b_2 .

Same Logic, More Neurons

The matrix equation stays the same:

$$Z_2 = W_2 A_1 + b_2$$

Let's say we now have **three neurons** in the second layer. The output becomes:

$$A_2 = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = a(Z_2) = a \left(\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \right) = \begin{bmatrix} a(z_1) \\ a(z_2) \\ a(z_3) \end{bmatrix}$$

Each z_i is computed using its own weights and bias:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Reminder

Each row of W_2 corresponds to one neuron in the layer. Each neuron gets the full input A_1 but transforms it differently.

10.1 Visual Recap

Here's what we've just built — a second layer with three neurons:

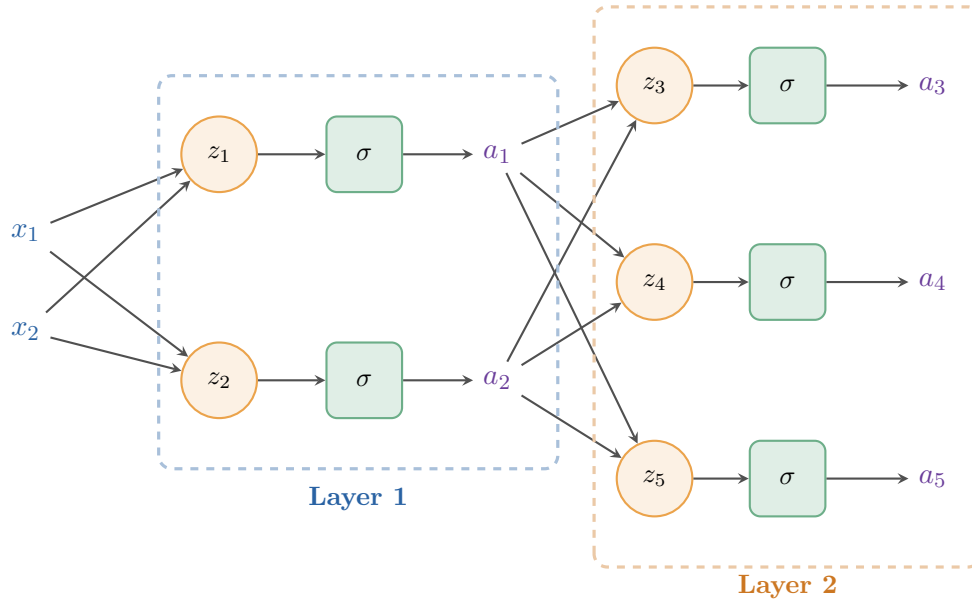


Figure 5: Two-layer neural network: 2 neurons in Layer 1, 3 neurons in Layer 2.

11 Adding a Layer

Key distinction:

- To add a **neuron within a layer**, you simply add a new **row** to the weight matrix and a new **entry** to the bias vector.
- To add a **new layer**, you introduce an entirely new **weight matrix** and a new **bias vector**.

12 Output Layer and Final Output

The **output layer** is the last layer of the neural network. It takes as input the output of the last hidden layer — and produces the final prediction.

Let's say we use the following weight matrix and bias vector:

$$W_3 = \begin{bmatrix} w_{11} & w_{12} & w_{13} \end{bmatrix} \quad \text{and} \quad b_3 = \begin{bmatrix} b_1 \end{bmatrix}$$

We compute the pre-activation of the output neuron:

$$Z_3 = \begin{bmatrix} z_1 \end{bmatrix} = W_3 A_2 + b_3 = \begin{bmatrix} w_{11} & w_{12} & w_{13} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \begin{bmatrix} b_1 \end{bmatrix}$$

Then we apply the activation function (here: sigmoid) to get the final prediction:

$$A_3 = a(Z_3) = \left[\frac{1}{1+e^{-z_1}} \right]$$

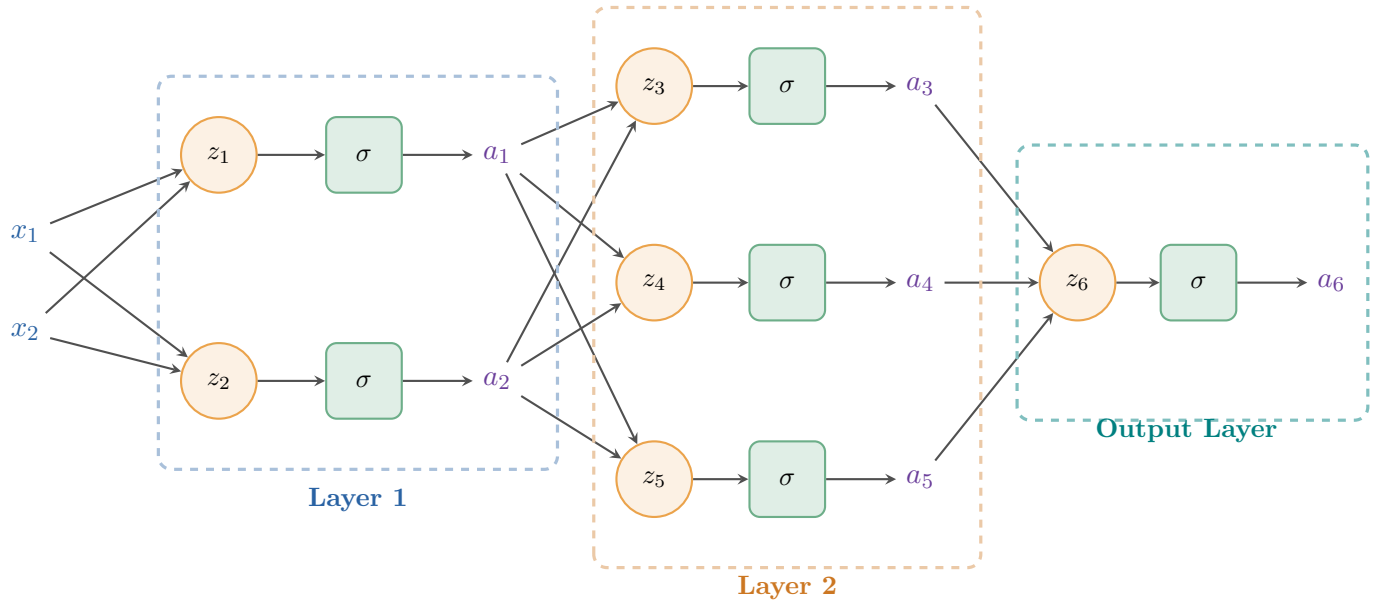


Figure 6: Complete neural network: 2 inputs, 2 hidden layers, 1 output neuron.

13 Summary

- **Adding a neuron** = add a row to W , and an element to b .
- **Adding a layer** = create a new W_i and b_i .

Each layer performs the same core sequence:

Layer Computation — General Rule

Input: A_{i-1}

Linear combination: $Z_i = W_i A_{i-1} + b_i$

Activation: $A_i = a(Z_i)$

Why matrices? Because they allow us to compute all neurons' operations at once — fast, in parallel, and efficiently on modern hardware.

14 Forward Propagation

Forward propagation is the process of passing the input through the entire network — from the input layer, through all hidden layers, to the final output.

At each step, the same logic applies:

- Apply a **linear operation** ($Z = WA + b$)
- Apply an **activation function** ($A = \sigma(Z)$)

This continues until we reach the final output:

$$A_3 = \text{network prediction}$$

Checkpoint — What We've Built So Far

Take a breath. You've just built a complete neural network from the ground up. Here's what you now understand:

- A **layer** is a collection of neurons operating in parallel: $Z = Wx + b$
- An **activation function** introduces non-linearity: $A = \sigma(Z)$
- Stacking layers creates a **deep network**: $x \rightarrow A_1 \rightarrow A_2 \rightarrow A_3$
- **Forward propagation** pushes data through the entire network to produce a prediction

Next: we go backward — to understand how the network *learns*.

15 Training via Backpropagation

Once we compute the output, we want to train the network — that is, adjust the weights and biases to reduce the prediction error.

To train the network, we follow these steps:

1. Define a **loss function** (such as log-loss)
2. Compute the **gradient of the loss** with respect to each parameter
3. Update the weights and biases using **gradient descent**

From our earlier notebook (for a single neuron), we saw:

$$\frac{\partial \mathcal{L}}{\partial w_1} = -\frac{x_1}{m} \sum_i (y_i - a_i) \quad \frac{\partial \mathcal{L}}{\partial w_2} = -\frac{x_2}{m} \sum_i (y_i - a_i) \quad \frac{\partial \mathcal{L}}{\partial b} = -\frac{1}{m} \sum_i (y_i - a_i)$$

Remember, we derived these expressions using the chain rule.

Now We Scale Up

We will do the exact same thing here — but with **matrices**.

15.1 Goal: Compute the Gradients

Let's focus on the following parameters:

- W_1, b_1 — parameters of the first layer
- W_2, b_2 — parameters of the second layer

Our goal is to understand how much each parameter influences the loss function. In other words, we want to compute:

$$\frac{\partial \mathcal{L}}{\partial W_1}, \quad \frac{\partial \mathcal{L}}{\partial b_1}, \quad \frac{\partial \mathcal{L}}{\partial W_2}, \quad \frac{\partial \mathcal{L}}{\partial b_2}$$

As before — like we did in the perceptron model — we will use the **chain rule**. This time, however, we will apply it step by step through the layers of a deep network.

Before We Start

Read carefully. This is the foundation of the entire backward pass.

You don't need to memorize equations — but you must understand the logic. It's all about how the functions are connected, and how parameters influence outputs.

If you get that, you'll understand every neural network.

15.2 Summary of the Network Functions

First Layer:

- Linear transformation:

$$Z_1 = W_1X + b_1$$

- Activation:

$$A_1 = \frac{1}{1 + e^{-Z_1}}$$

Second Layer (Last Hidden Layer):

- Linear transformation:

$$Z_2 = W_2A_1 + b_2$$

Important: Here, the input is A_1 , not the original input X .

- Activation:

$$A_2 = \frac{1}{1 + e^{-Z_2}}$$

- Loss function:

$$\mathcal{L} = -\frac{1}{m} \sum (y \log A_2 + (1 - y) \log(1 - A_2))$$

We compute the loss using A_2 , the output of the last layer — not A_1 .

16 Important

Finding the gradients means identifying how the loss $\mathcal{L}$ depends on each parameter of the network.

But as you've already seen, the **entry point** between the loss $\mathcal{L}$ and the rest of the network is the output of the last layer — namely, A_2 .

So even if we want to compute the gradients of the parameters in the *first layer*, we must first pass through the *second layer*.

Key Insight

This is the core principle of backpropagation:

We must go backward through the network — layer by layer — to compute the gradients.

And when you think about it, it actually makes perfect sense. This is the whole point of deep learning: the model builds up its complexity layer after layer. So naturally, we need to go back down that stack to understand how each parameter affects the final outcome.

17 Gradients for the Layer Just Before the Output

Let's now compute the gradients for the second layer, using the chain rule.

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2}$$

Let's break this down:

- $\frac{\partial \mathcal{L}}{\partial A_2}$ is the gradient of the loss with respect to the network output
- $\frac{\partial A_2}{\partial Z_2}$ is the derivative of the activation function (sigmoid)
- $\frac{\partial Z_2}{\partial W_2}$ and $\frac{\partial Z_2}{\partial b_2}$ are the local gradients of the model

Since the first two terms appear identically in both equations, we define:

$$dZ_2 = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

Then the gradients simplify to:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= dZ_2 \cdot \frac{\partial Z_2}{\partial W_2} \\ \frac{\partial \mathcal{L}}{\partial b_2} &= dZ_2 \cdot \frac{\partial Z_2}{\partial b_2}\end{aligned}$$

18 For the First Layer

Let's now focus on computing the gradients with respect to the parameters of the **first layer**.

Using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1}$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial b_1}$$

Where:

- $\frac{\partial \mathcal{L}}{\partial A_1}$ is the gradient of the loss with respect to the output of the first layer
- $\frac{\partial A_1}{\partial Z_1}$ is the derivative of the activation function
- $\frac{\partial Z_1}{\partial W_1}$ and $\frac{\partial Z_1}{\partial b_1}$ are the local gradients of the linear model

But — as we said before — there's a crucial observation here:

There is no direct link between the loss $\mathcal{L}$ and the output A_1 of the first layer. To compute this gradient, we must go through the second layer — just like the signal does in the forward pass.

Pause and Think

How do we relate the second layer to the first? What is the variable that connects both layers?

Take a moment and try to identify the link yourself.

The answer?

It's in the equation:

$$Z_2 = W_2 A_1 + b_2$$

This shows that A_1 is the input to the second layer. So to compute $\frac{\partial \mathcal{L}}{\partial A_1}$, we apply the chain rule again:

$$\frac{\partial \mathcal{L}}{\partial A_1} = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1}$$

What Just Happened

We found a way to compute a gradient that had no direct path to the loss.

There is no direct dependency between $\mathcal{L}$ and A_1 — but by passing through A_2 and Z_2 , we built the connection.

This is the heart of backpropagation: **working backward through the layers using the chain rule to connect everything.**

19 Full Gradient Expression for the First Layer

Let's now write the full expression for the gradients of the first layer, step by step, with the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1}$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial b_1}$$

Let's be honest: yes, that's a long chain. But this is how deep learning works. And no, it's not “just a few lines of code.” If it were simple, everyone would be doing it.

Side Note

Correction: everyone *is* a “data scientist” on LinkedIn. But we're here to understand what makes a **real and honest** one.

Still — we can simplify.

Just like we did earlier, we define:

$$dZ_1 = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1}$$

But since we already defined:

$$dZ_2 = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

We can write:

$$dZ_1 = dZ_2 \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1}$$

This gives us the final simplified expressions:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial W_1} \\ \frac{\partial \mathcal{L}}{\partial b_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial b_1} \end{aligned}$$

Structure of the Backward Pass

Each gradient is built from:

- the gradient from the next layer (dZ_2),
- the chain of derivatives that connects the current layer,
- and the local gradients of the linear model.

It's structured, logical, and beautifully recursive.

20 Summary of the Gradient Equations

Let's recap what we want to compute during the backward pass:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_2} &= dZ_2 \cdot \frac{\partial Z_2}{\partial W_2} \\ \frac{\partial \mathcal{L}}{\partial b_2} &= dZ_2 \cdot \frac{\partial Z_2}{\partial b_2} \\ \frac{\partial \mathcal{L}}{\partial W_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial W_1} \\ \frac{\partial \mathcal{L}}{\partial b_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial b_1} \end{aligned}$$

To get there, we use:

$$\begin{aligned} dZ_2 &= \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \\ dZ_1 &= dZ_2 \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \end{aligned}$$

Strategy:

1. Compute dZ_2 from the output layer.
2. Use it to compute $\frac{\partial \mathcal{L}}{\partial W_2}$ and $\frac{\partial \mathcal{L}}{\partial b_2}$.
3. Backpropagate to get dZ_1 .

4. Use it to compute $\frac{\partial \mathcal{L}}{\partial W_1}$ and $\frac{\partial \mathcal{L}}{\partial b_1}$.

21 Let's Get to the Point

In theory, things are clear. In practice, there are no shortcuts.

If we want to understand how a neural network learns, we have to compute every gradient — one by one.

This is the work.

And now, we're ready to do it.

21.1 Computing dZ_2

Let's begin with the calculation of dZ_2 — because this term appears in both gradient expressions for the second layer, and we also need it to backpropagate further to the first layer.

We define:

$$dZ_2 = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

Where:

- $\frac{\partial \mathcal{L}}{\partial A_2}$ is the gradient of the loss with respect to the output of the network,
- $\frac{\partial A_2}{\partial Z_2}$ is the derivative of the sigmoid activation.

Let's compute the first term — the derivative of the loss.

$$\frac{\partial \mathcal{L}}{\partial A_2} = ?$$

Wait... you don't remember this?

Come on — it's the same expression we derived earlier in the previous notebook using the log-loss function. Fine. I'll recap it but after that, you're on your own. I'm not your mom. For goodness' sake.

Loss function (log-loss):

$$\mathcal{L} = -\frac{1}{m} \sum_i (y_i \log A_2 + (1 - y_i) \log(1 - A_2))$$

We compute its derivative with respect to A_2 :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial A_2} &= -\frac{1}{m} \sum_i \left(y_i \cdot \frac{1}{A_2} - (1 - y_i) \cdot \frac{1}{1 - A_2} \right) \\ &= -\frac{1}{m} \sum_i \left(\frac{y_i}{A_2} - \frac{1 - y_i}{1 - A_2} \right) \\ &= -\frac{1}{m} \sum_i \left(\frac{y_i(1 - A_2) - (1 - y_i)A_2}{A_2(1 - A_2)} \right) \\ &= -\frac{1}{m} \sum_i \left(\frac{y_i - A_2}{A_2(1 - A_2)} \right) \end{aligned}$$

So yes — this is the final simplified form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial A_2} = -\frac{1}{m} \sum_i \left(\frac{y_i - A_2}{A_2(1 - A_2)} \right)}$$

We now compute the second term in:

$$dZ_2 = \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

Namely:

$$\frac{\partial A_2}{\partial Z_2} = A_2 \cdot (1 - A_2)$$

No way... you forgot this one too?

All right — I said I wouldn't hold your hand anymore, but let's be honest — this notebook is meant to be your guide.

So let's recap.

The activation function introduces non-linearity into the network, allowing it to model complex relationships between inputs and outputs.

The sigmoid activation is defined by:

$$a(z) = \frac{1}{1 + e^{-z}}$$

We now compute its derivative:

$$\begin{aligned} \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right) &= \frac{-1}{(1 + e^{-z})^2} \cdot \frac{d}{dz} (1 + e^{-z}) \\ &= \frac{-1}{(1 + e^{-z})^2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \cdot \left(\frac{e^{-z}}{1 + e^{-z}} \right) \\ &= a(z) \cdot (1 - a(z)) \end{aligned}$$

So for any neuron output z , the derivative becomes:

$$\boxed{\frac{da(z)}{dz} = a(z)(1 - a(z))}$$

Hence, for the entire vector A_2 , we apply this element wise:

$$\frac{\partial A_2}{\partial Z_2} = \begin{bmatrix} a(z_1)(1 - a(z_1)) \\ a(z_2)(1 - a(z_2)) \\ a(z_3)(1 - a(z_3)) \end{bmatrix} \quad \text{or simply: } A_2 \cdot (1 - A_2)$$

21.1.1 Final Expression for dZ_2

We now write the full expression for the error signal at the output layer:

$$\begin{aligned} dZ_2 &= \frac{\partial \mathcal{L}}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \\ &= -\frac{1}{m} \sum_i \left(\frac{y_i - A_2}{A_2(1 - A_2)} \right) \cdot A_2 \cdot (1 - A_2) \\ &= -\frac{1}{m} \sum_i (y - A_2) \\ &= A_2 - y \end{aligned}$$

Why is $\frac{1}{m} \sum_i (A_2 - y) = A_2 - y$ legitimate here?

Because we are, at this stage, working with a single training example. In that context, summing over i yields only one term — so the average is the term itself. No division by m is needed.

However: the averaging over multiple samples is not forgotten. **It will be accounted for during parameter updates.**

Result — Output Layer Error

$$dZ_2 = A_2 - y$$

This expression — the difference between prediction and truth — is the entry point of the backward pass.

21.2 Computing $\frac{\partial \mathcal{L}}{\partial W_2}$ and $\frac{\partial \mathcal{L}}{\partial b_2}$

Once dZ_2 is computed, we can express the gradients of the loss with respect to the parameters of the second layer:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_2} = dZ_2 \cdot \frac{\partial Z_2}{\partial W_2}} \quad \text{and} \quad \boxed{\frac{\partial \mathcal{L}}{\partial b_2} = dZ_2 \cdot \frac{\partial Z_2}{\partial b_2}}$$

I want you to look at the different equations we highlighted before to understand the relation between the different variables and find the wanted gradients.

If you look at the equations, you will see that we can find these gradients using the second layer model:

$$Z_2 = W_2 A_1 + b_2$$

We will now compute each of these terms explicitly.

$$\frac{\partial Z_2}{\partial W_2} = A_1$$

where A_1 is the output of the previous layer.

Similarly, we write the expression for the bias term:

$$\frac{\partial Z_2}{\partial b_2} = 1$$

This corresponds to the derivative of the affine shift with respect to its own scalar. Substituting into the chain rule:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= dZ_2 \cdot A_1 \\ \frac{\partial \mathcal{L}}{\partial b_2} &= dZ_2 \cdot 1\end{aligned}$$

Where:

- $dZ_2 = A_2 - y$ is the gradient from the output layer,
- A_1 is the activation of the previous layer.

Now we recover the factor $\frac{1}{m}$ that we omitted earlier:

- The summation is implicitly handled by the matrix product $dZ_2 \cdot A_1^T$, since we are using vectorized notation.
- For the bias — which is scalar or vector-valued per neuron — we sum the components of dZ_2 explicitly.

Thus, the final expressions are:

Layer 2 Gradients

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= \frac{1}{m} (A_2 - y) \cdot A_1^T \\ \frac{\partial \mathcal{L}}{\partial b_2} &= \frac{1}{m} \sum_m (A_2 - y)\end{aligned}$$

21.3 Computing dZ_1

We now compute dZ_1 , which is required to obtain the gradients of the first layer:

$$\frac{\partial \mathcal{L}}{\partial W_1} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b_1}$$

The expression for dZ_1 comes from the chain rule applied backward:

$$dZ_1 = dZ_2 \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1}$$

Where:

- dZ_2 is the gradient of the loss with respect to the pre-activation output Z_2 ,
- $\frac{\partial Z_2}{\partial A_1} = W_2$ — from the layer equation $Z_2 = W_2 A_1 + b_2$,
- $\frac{\partial A_1}{\partial Z_1} = A_1 \cdot (1 - A_1)$ — the derivative of the sigmoid activation.

So we write:

$$dZ_1 = (dZ_2 \cdot W_2) \circ A_1 \cdot (1 - A_1)$$

where $\circ$ denotes the element-wise (Hadamard) product.

Interpretation

We are propagating the gradient from layer 2 through the weights W_2 , modulating it by the sensitivity of the activations A_1 — just as we did in the output layer.

21.4 Computing $\frac{\partial \mathcal{L}}{\partial W_1}$ and $\frac{\partial \mathcal{L}}{\partial b_1}$

Now with dZ_1 , we compute:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial W_1} \\ \frac{\partial \mathcal{L}}{\partial b_1} &= dZ_1 \cdot \frac{\partial Z_1}{\partial b_1} \end{aligned}$$

From the model:

$$Z_1 = W_1 X + b_1$$

we derive:

$$\frac{\partial Z_1}{\partial W_1} = X \quad \text{and} \quad \frac{\partial Z_1}{\partial b_1} = 1$$

Where:

- X is the input data matrix,
- 1 represents a constant vector aligned with the bias shape.

We can now express the raw gradients:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_1} &= dZ_1 \cdot X \\ \frac{\partial \mathcal{L}}{\partial b_1} &= dZ_1 \cdot 1 \end{aligned}$$

with dZ_1 being the gradient of the loss with respect to the pre-activation values Z_1 .

As before, we must include the normalization factor $\frac{1}{m}$, and sum over samples for the bias term.

Final vectorized expressions:

Layer 1 Gradients

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_1} &= \frac{1}{m} dZ_1 \cdot X^\top \\ \frac{\partial \mathcal{L}}{\partial b_1} &= \frac{1}{m} \sum_m dZ_1 \end{aligned}$$

Where:

$$dZ_1 = (dZ_2 \cdot W_2) \circ A_1 \cdot (1 - A_1)$$

is the gradient of the loss function with respect to the output of the first layer.

Key Insight

All gradients are now expressed as matrix products or elementwise operations — enabling efficient and scalable backpropagation.

22 Summary of the Gradients

Here it is.

No more equations, no more calculus — the math is done. We now have everything we need to update the parameters of our neural network.

Final gradient expressions:

Final Gradient Equations

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{1}{m} (A_2 - y) \cdot A_1^\top$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{1}{m} \sum_m (A_2 - y)$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{1}{m} dZ_1 \cdot X^\top$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \frac{1}{m} \sum_m dZ_1$$

Intermediate expressions:

Intermediate Quantities

$$dZ_2 = A_2 - y$$

$$dZ_1 = (dZ_2 \cdot W_2) \circ A_1 \cdot (1 - A_1)$$

I know what you're thinking:

"Those equations are ugly as hell."

But here's the truth:

1. I don't care — this is the real stuff. We're not at a fashion show.
2. You only need to do this **once**.

A data scientist isn't a mathematician. They don't compute gradients by hand. They build systems, they automate, they abstract.

But — they need to understand what happens under the hood. That's why you've done the work.

And now, you're ready.

You've gone through every step. No shortcuts, no magic. Just honest computation.

Next Stop — Practice

Let's code this network from scratch.

It'll be shockingly easy — because the math is already done.

Meet me in the next episode.

Deep Learning From Scratch

An open-source learning journey.

GitHub: <https://github.com/Pchambet/Deep-Learning-from-Scratch>

LinkedIn: <https://www.linkedin.com/in/pierre-chambet/>

© 2026 Pierre Chambet